Capital University

Faculty of Computer Science & Artificial Intelligence

Priority vs SRTF Comparison Project

CS 241 Operating Systems-1 – Spring "Semester 2" 2025-2026

(C1)

1.Project Overview

The "Priority-vs-SRTF" project is an interactive educational tool designed to simulate and compare two fundamental CPU scheduling algorithms:

- ***Priority Scheduling***: A preemptive algorithm where each process is assigned a priority value. Processes with the highest priority (represented by lower integer values in this context) are executed first.
- ***SRTF (Shortest Remaining Time First):*** A preemptive algorithm that always executes the process with the shortest remaining burst time, effectively **minimizing the overall wait times** for the system.

This project features a Graphical User Interface (GUI) that allows users to input process datasets, visualize timelines via Gantt charts, and calculate essential performance metrics such as Average *Waiting Time (WT)*, *Turnaround Time (TAT)*, and *Response Time (RT)* to directly compare the two algorithms.

2.Test Scenarios

Scenario A: Basic Mixed Workload

A standard workload with five processes, varied arrival times, burst times, and priorities. The purpose is to observe normal scheduler behavior under typical conditions.

Input:

Process ID	Arrival Time	Burst Time	Priority
P1	0	3	2
P2	1	2	1
P3	2	4	3
P4	3	1	2
P5	4	3	1

Results:

Priority Scheduling

Process ID	WT	TAT	RT
P1	5	8	0
P2	0	2	0

P3	7	11	7
P4	5	6	5
P5	0	3	0

SRTF

Process ID	WT	TAT	RT
P1	0	3	0
P2	3	5	3
P3	7	11	7
P4	0	1	0
P5	2	5	2

Comparison - Scenario A:

Metric	Priority	SRTF	Winner
Avg WT	3.40	2.40	**SRTF**
Avg TAT	6.00	5.00	**SRTF**
Avg RT	2.40	2.40	**Tie**

Observation:

SRTF produced a *lower average waiting time (2.40 vs 3.40)* and *lower average turnaround time*. Priority Scheduling caused higher waiting for low-priority processes such as P3, while SRTF favoured shorter jobs regardless of priority.

Scenario B : Conflict Between Priority and Burst Time

This scenario is designed to expose the key difference between the two algorithms. P1 has the highest priority (1) but the longest burst time (10). All other processes have short burst times but lower priority.

Input:

Process ID	Arrival Time	Burst Time	Priority
P1	0	10	1
P2	1	1	5
P3	2	1	4
P4	3	2	3
P5	4	1	5

Results:

Priority Scheduling

Process ID	WT	TAT	RT
P1	0	10	0
P2	12	13	12
P3	10	11	10
P4	7	9	7
P5	10	11	10

SRTF

Process ID	WT	TAT	RT
P1	5	15	0
P2	0	1	0
P3	0	1	0
P4	0	2	0
P5	1	2	1

Comparison - Scenario B:

Metric	Priority	SRTF	Winner
Avg WT	7.80	1.20	**SRTF**
Avg TAT	10.80	4.20	**SRTF**
Avg RT	7.80	0.20	**SRTF**

Observation:

This scenario clearly shows the trade-off. Priority Scheduling kept P1 running for its full 10 units, causing all other processes to wait a long time. SRTF preempted P1 immediately when shorter jobs arrived, giving them near-zero waiting times. **SRTF was significantly more efficient here (Avg WT: 1.20 vs 7.80)**.

Scenario C: Starvation-Sensitive Case

This scenario is designed to reveal starvation risk. P1 and P2 have low priority (3) with long burst times. P3 and P4 have high priority (1) with short burst times. P5 has low priority and a very long burst time.

Input:

Process ID	Arrival Time	Burst Time	Priority
P1	0	10	3
P2	2	12	3
P3	4	3	1
P4	6	5	1
P5	8	15	3

Results:

Priority Scheduling

Process ID	WT	TAT	RT
P1	8	18	0
P2	16	28	16
P3	0	3	0
P4	1	6	1
P5	22	37	22

SRTF

Process ID	WT	TAT	RT
P1	8	18	0
P2	16	28	16
P3	0	3	0
P4	1	6	1
P5	22	37	22

Comparison - Scenario C:

Metric	Priority	SRTF	Winner
Avg WT	9.40	9.40	**Tie**
Avg TAT	18.40	18.40	**Tie**
Avg RT	7.80	7.80	**Tie**

Observation:

In this workload, both algorithms produced identical results because the high-priority short processes (P3, P4) were also the shortest jobs. This means both Priority and SRTF made the same scheduling decisions. However, P5 waited 22 units, demonstrating starvation risk when long low-priority jobs compete with shorter ones.

3.Validation

validation is implemented at the input level - the fields only accept valid numeric values, so invalid data cannot be entered in the first place. This is a proactive approach to input safety.

Invalid Input	How the System Handles It
Negative Arrival Time	Field does not accept negative numbers
Zero or Negative Burst Time	Field does not accept zero or negative numbers
Negative Priority	Field does not accept negative numbers
Letters or symbols in any field	Field only accepts numeric values

4. Required Analysis Questions

Q1: Which algorithm produced the lower average waiting time?

SRTF produced the lower average waiting time in Scenarios A and B. In Scenario A, SRTF achieved Avg WT = 2.40 compared to Priority's 3.40. In Scenario B, the difference was dramatic — SRTF: 1.20 vs Priority: 7.80. Scenario C was a tie at 9.40.

Q2: Which algorithm produced the lower average response time?

SRTF produced the lower average response time overall. In Scenario B, SRTF achieved an exceptional Avg RT of 0.20 compared to Priority's 7.80, because short jobs were served almost immediately. In Scenario A both algorithms tied at 2.40.

Q3: Did priority values improve treatment of urgent processes?

Yes. In Scenario B, P1 (Priority 1) was served immediately by Priority Scheduling and ran without interruption for its full 10 units. This benefited the urgent process directly. However, it severely penalised all other processes, causing them to wait 10+ units. Priority Scheduling is effective for real-time systems where certain tasks must be treated with urgency.

Q4: Did SRTF favor short jobs more aggressively?

Yes. SRTF is inherently biased toward short jobs. In Scenario B, processes P2, P3, P4, and P5 all had waiting times of 0 or 1 under SRTF, while P1 (burst=10) was repeatedly preempted and waited 5 units. SRTF minimises average waiting time but can cause starvation for long processes.

Q5: Which algorithm would you recommend for the tested workload, and why?

For general-purpose workloads (Scenario A), SRTF is recommended because it consistently produces lower average waiting and turnaround times. For workloads where process urgency matters (real-time systems), Priority Scheduling is more appropriate because it guarantees that critical tasks are served first regardless of burst length.

5. Final Conclusion

5.1 Which algorithm performed better overall

SRTF performed better on average metrics across Scenarios A and B. It achieved lower Avg WT and Avg TAT in both scenarios. In Scenario C both algorithms were equal. SRTF wins on raw performance in most workloads.

5.2 Which metrics were better under each algorithm?

Priority Scheduling	SRTF
Better RT for high-priority urgent processes Guaranteed service order by importance	Better Avg WT in most workloads Better Avg TAT overall Better Avg RT for short jobs

5.3 Main Trade-off Observed

The core trade-off is between urgency and efficiency. Priority Scheduling guarantees that the most important process runs first - this is critical in real-time and safety-critical systems. However, it can severely delay other processes, especially if high-priority jobs have long burst times. SRTF minimizes average waiting time and is more efficient overall, but it completely ignores process urgency and can starve long processes indefinitely.

5.4 Which algorithm appeared fairer in practice ?

SRTF appeared fairer in practice. It distributes CPU time based on remaining work rather than an assigned label, meaning no process is artificially delayed due to its priority number. All processes receive CPU time proportional to how much work they need. Priority Scheduling, while useful for critical systems, is inherently unfair to low-priority processes and can cause starvation.